

华东师范大学软件工程学院数据挖掘期末报告

课程名称：数据挖掘

修读时间：

2025 年秋季

小组成员及学号：俞骞(10235101488)、丁乙(10235101489)、张建夫(10235101477)

项目名称：CSIRO - Image2Biomass Prediction

一、课题背景介绍

草地生态系统是全球陆地生态系统的重要组成部分，其生物量的准确估测对于畜牧业管理、碳循环研究和生态环境监测具有重要意义。传统的生物量测量方法主要依赖于野外采样，这种方法不仅耗时费力，而且难以实现大范围的动态监测。

随着遥感技术的发展，利用卫星或无人机获取的光谱图像进行生物量反演已成为主流研究方向。本课题基于CSIRO（澳大利亚联邦科学与工业研究组织）提供的草地遥感图像数据集，旨在开发一个基于深度学习的高精度生物量预测模型。该模型需要同时预测五个关键生物量指标：Dry_Clover_g（干三叶草质量），Dry_Dead_g（干枯草质量），Dry_Green_g（绿草质量），Dry_Total_g（干草总质量），GDM_g（地上生物量干重）

本项目的挑战在于如何从有限的高分辨率图像中提取有效特征，并准确预测多个高度相关的生物量指标，同时处理数据集中存在的小样本、类别不平衡和缺失值等问题。

二、小组分工情况

- 俞骞：模型结构设计，实验验证工作
- 丁乙：数据划分策略, 过拟合对抗系统设计, PPT编排
- 张建夫：预训练模型选择

三、数据挖掘算法设计

3.1 数据预处理

数据预处理流程是训练的关键基础，涉及多个关键步骤，旨在将原始异构数据转化为适合深度学习模型训练的规范化格式。具体数据预处理的核心流程如下：

1. 数据格式转换：将训练数据从长格式转换为宽格式•俞骞
2. 缺失值处理：对连续变量进行中位数填充•俞骞
3. 特征标准化：对连续变量进行 Z-score 标准化•俞骞
4. 分类变量编码：对类别变量进行标签编码•俞骞
5. 数据集划分：采用 GroupKFold 进行交叉验证划分•丁乙
6. 数据增强：训练阶段应用多种图像增强技术•丁乙

3.1.1 数据格式转换与结构化

原始训练数据采用**长格式 (long format)** 存储（即每行数据代表一个样本在某个特定生物量目标（如干草总质量、绿草质量等）上的观测值）。这种格式便于数据收集，但不适合直接用于模型训练。因此首先将长格式转换为**宽格式 (wide format)**。

算法实现流程如下：

从 sample_id 中提取 image_id 作为唯一标识，使用 pivot_table 将 5 个生物量目标（Dry_Clover_g、Dry_Dead_g、Dry_Green_g、Dry_Total_g、GDM_g）从长格式转换为宽格式，最后合并元数据信息，减少数据冗余

```
01 def pivot_train_data(train_df):
02     """ 将训练数据从长格式转换为宽格式 """
03     train_df['image_id'] = train_df['sample_id'].apply(extract_image_id)
04
05     # 透视目标值
06     targets_wide = train_df.pivot_table(
07         index='image_id',
08         columns='target_name',
09         values='target',
10         aggfunc='first'
11     ).reset_index()
12
13     # 合并元数据
14     metadata_cols = ['image_id', 'image_path', 'Sampling_Date', 'State', 'Species',
15                     'Pre_GSHH_NDVI', 'Height_Ave_cm']
16     metadata = train_df[metadata_cols].drop_duplicates('image_id').reset_index(drop=True)
17
18     # 合并透视结果和元数据
19     df_wide = metadata.merge(targets_wide, on='image_id', how='left')
20
21     return df_wide
```

3.1.2 缺失值处理

本网络针对不同类型的特征采用了差异化的填补策略，确保数据完整性同时避免

引入偏差。

算法实现流程如下：

对连续型特征 `Pre_GSHH_NDVI` 和 `Height_Ave_cm`，分别计算其在训练集中的中位数，并使用该中位数填充训练集和验证集中的缺失值，确保数据分布的稳定性。

```
def fit_and_transform_fold_data(train_data, val_data, categorical_cols=['State', 'Species']):  
    """  
    在训练数据上拟合预处理对象（标准化器、编码器）  
    并将相同的转换应用于验证数据  
    """  
  
    scalers = {}  
    label_encoders = {}  
  
    # 缺失值填充  
    print("应用缺失值填充和标准化...")  
  
    # 仅从训练数据计算中位数  
    ndvi_median = train_data['Pre_GSHH_NDVI'].median()  
    height_median = train_data['Height_Ave_cm'].median()  
  
    # 将训练集的中位数应用于两个数据集  
    train_data['Pre_GSHH_NDVI'].fillna(ndvi_median, inplace=True)  
    val_data['Pre_GSHH_NDVI'].fillna(ndvi_median, inplace=True)  
    train_data['Height_Ave_cm'].fillna(height_median, inplace=True)  
    val_data['Height_Ave_cm'].fillna(height_median, inplace=True)  
  
    # ... 后续处理省略
```

3.1.3 特征标准化

不同特征的量纲和数值范围差异巨大，直接输入模型会导致训练不稳定。我们实施了分层标准化策略，消除特征间的量纲差异。

算法实现流程如下：

对连续型特征 `Pre_GSHH_NDVI` 和 `Height_Ave_cm` 分别使用 `StandardScaler` 进行 Z-score 标准化，标准化参数仅从训练集学习，验证集使用相同的转换参数，防止数据泄露。

```
def fit_and_transform_fold_data(train_data, val_data, categorical_cols=['State', 'Species']):  
    # ... 缺失值填充部分  
  
    # 标准化  
    # 在训练数据上拟合StandardScaler  
    ndvi_scaler = StandardScaler()  
    train_data['ndvi_scaled'] = ndvi_scaler.fit_transform(train_data[['Pre_GSHH_NDVI']])  
  
    # 将学习到的转换（无需重新拟合）应用于验证数据  
    val_data['ndvi_scaled'] = ndvi_scaler.transform(val_data[['Pre_GSHH_NDVI']])  
    scalers['ndvi'] = ndvi_scaler # 保存供测试集预测使用  
  
    height_scaler = StandardScaler()  
    train_data['height_scaled'] = height_scaler.fit_transform(train_data[['Height_Ave_cm']])  
    val_data['height_scaled'] = height_scaler.transform(val_data[['Height_Ave_cm']])  
    scalers['height'] = height_scaler  
  
    # ... 后续处理省略
```

3.1.4 分类变量编码

机器学习模型无法直接处理文本型类别数据，需要进行数值编码以适配模型输入要求。

算法实现流程如下：

对 State 和 Species 类别特征使用 LabelEncoder 进行整数编码。编码映射关系仅从训练集学习，验证集中出现的新类别通过安全转换机制映射到特殊索引，避免运行时错误。

```
def fit_and_transform_fold_data(train_data, val_data, categorical_cols=['State', 'Species']):
    # ... 缺失值填充和标准化部分

    # 编码
    for col in categorical_cols:
        le = LabelEncoder()

        # 仅在训练数据的唯一值上拟合
        le.fit(train_data[col].astype(str).unique())

        # 转换训练数据
        train_data[f'{col}_encoded'] = le.transform(train_data[col].astype(str))

        # 安全地转换验证数据: 处理仅在验证集中存在的类别
        def safe_transform(s):
            # 创建已知类别到索引的映射字典
            mapping = {cls: idx for idx, cls in enumerate(le.classes_)}
            # 将未见类别设置为新索引
            unseen_idx = len(le.classes_)
            return [mapping.get(x, unseen_idx) for x in s]

        val_data[f'{col}_encoded'] = safe_transform(val_data[col].astype(str))

        label_encoders[col] = le

    return train_data, val_data, scalers, label_encoders
```

3.1.5 数据划分策略

为避免过拟合并提高模型泛化能力，我们采用了分组交叉验证策略，确保同一图像的不同观测不会同时出现在训练集和验证集中。GroupKFold 的核心原理是组内不分离（确保同一组（同一图像）的所有样本要么都在训练集，要么都在验证集）、防止数据泄露（避免模型在训练时“看到”验证集中的相关信息）、更真实的评估（模拟真实场景中新图像的预测性能）

算法实现流程如下：

使用 GroupKFold 按 image_id 进行 5 折交叉验证划分，每次使用 4 份作为训练集，1 份作为验证集，确保图像级别的数据独立性。

```
def train_model():
    # ... 数据加载和预处理

    # 使用GroupKFold进行交叉验证（按图像ID分组）
    gkf = GroupKFold(n_splits=Config.NUM_FOLDS)
    groups = train_wide['image_id'].values

    folds_to_train = range(Config.NUM_FOLDS) if Config.TRAIN_FOLD == -1 else [Config.TRAIN_FOLD]

    # 遍历每个折进行训练
    for fold_idx, (train_idx, val_idx) in enumerate(gkf.split(train_wide, groups=groups)):
        if fold_idx not in folds_to_train:
            continue

        train_data_raw = train_wide.iloc[train_idx].reset_index(drop=True)
        val_data_raw = train_wide.iloc[val_idx].reset_index(drop=True)

    # ... 后续流程省略
```

3.1.6 数据增强技术

为增加数据多样性并防止过拟合，本网络实施了多种数据增强策略，除了传统的数据增强策略（随机缩放裁剪、随机翻转、随机旋转、颜色抖动等），还有 Mixup 数据增强，模拟不同条件下的图像变化，提高模型鲁棒性。Mixup 是一种创新的数据增强方法，其核心公式是

$$\begin{aligned}\tilde{x} &= \lambda x_i + (1 - \lambda) x_j \\ \tilde{y} &= \lambda y_i + (1 - \lambda) y_j\end{aligned},$$

主要优势在于

1. 正则化效果：鼓励模型学习更平滑的决策边界
2. 标签平滑：创建"软标签"，减少模型过度自信
3. 提高鲁棒性：增强模型对输入微小变化的稳定性

算法实现流程如下：

在训练阶段应用随机缩放裁剪、水平/垂直翻转、旋转以及颜色调整等多种图像增强技术，同时实现 Mixup 数据混合策略，进一步提高模型泛化能力。

```

def get_transforms(train=True):
    if train:
        return transforms.Compose([
            # 调整大小/裁剪
            transforms.RandomResizedCrop(
                Config.IMG_SIZE,
                scale=(0.9, 1.0), # 仅轻微缩放变化
                ratio=(0.95, 1.05) # 保持接近正方形
            ),

            # 几何增强
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomVerticalFlip(p=0.5),
            transforms.RandomRotation(30),

            # 颜色增强
            transforms.ColorJitter(
                brightness=0.2,
                contrast=0.2,
                saturation=0.2,
                hue=0.05
            ),

            # 转换和归一化
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ])
    else:
        return transforms.Compose([
            transforms.Resize((Config.IMG_SIZE, Config.IMG_SIZE)),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ])

def mixup_data(x, y, alpha=0.2):
    """Mixup数据增强"""
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1

    batch_size = x.size()[0]
    index = torch.randperm(batch_size).to(x.device)

    mixed_x = lam * x + (1 - lam) * x[index, :]
    y_a, y_b = y, y[index]

```

四、实验配置和数据集说明

4.1 数据集概述•俞骞

数据来源：CSIRO草地生物量数据集

- 总样本数：357个独特图像
- 图像格式：RGB高分辨率遥感图像
- 图像尺寸：原始尺寸不一，统一调整为 532×532
- 目标变量：5个连续生物量指标
- 辅助特征：NDVI指数、草高、地区、草种

数据划分：

- 训练集：285个样本（80%）

- 验证集：72个样本（20%）
- 5折交叉验证确保统计可靠性

4.2 实验环境配置•张建夫

硬件配置：

- GPU: NVIDIA Tesla V100 32GB
- CPU: Intel Xeon Gold 6248R
- 内存: 256GB DDR4
- 存储: 2TB NVMe SSD

软件环境：

- 操作系统: Ubuntu 20.04 LTS
- 深度学习框架: PyTorch 2.0.0
- Python版本: 3.9.18
- CUDA版本: 11.8
- 关键库: torchvision, scikit-learn, pandas, numpy, PIL

4.3 超参数配置•丁乙

python

class Config:

模型配置

 BACKBONE_MODEL = 'convnext_base' *# 主干网络*

 IMG_SIZE = 532 *# 输入图像尺寸*

训练配置

 BATCH_SIZE = 16 *# 批量大小*

 NUM_EPOCHS = 150 *# 最大训练轮数*

 LEARNING_RATE = 5e-5 *# 初始学习率*

 PATIENCE = 10 *# 早停耐心值*

 NUM_WORKERS = 4 *# 数据加载线程数*

交叉验证

NUM_FOLDS = 5 # 交叉验证折数

多任务学习

USE_MULTITASK = True # 启用多任务学习

AUXILIARY_WEIGHT = 0.3 # 辅助任务权重

数据增强

USE_MIXUP = True # 启用Mixup

MIXUP_ALPHA = 0.4 # Mixup参数

目标权重

TARGET_WEIGHTS = [0.1, 0.1, 0.1, 0.5, 0.2]

随机种子

SEED = 42

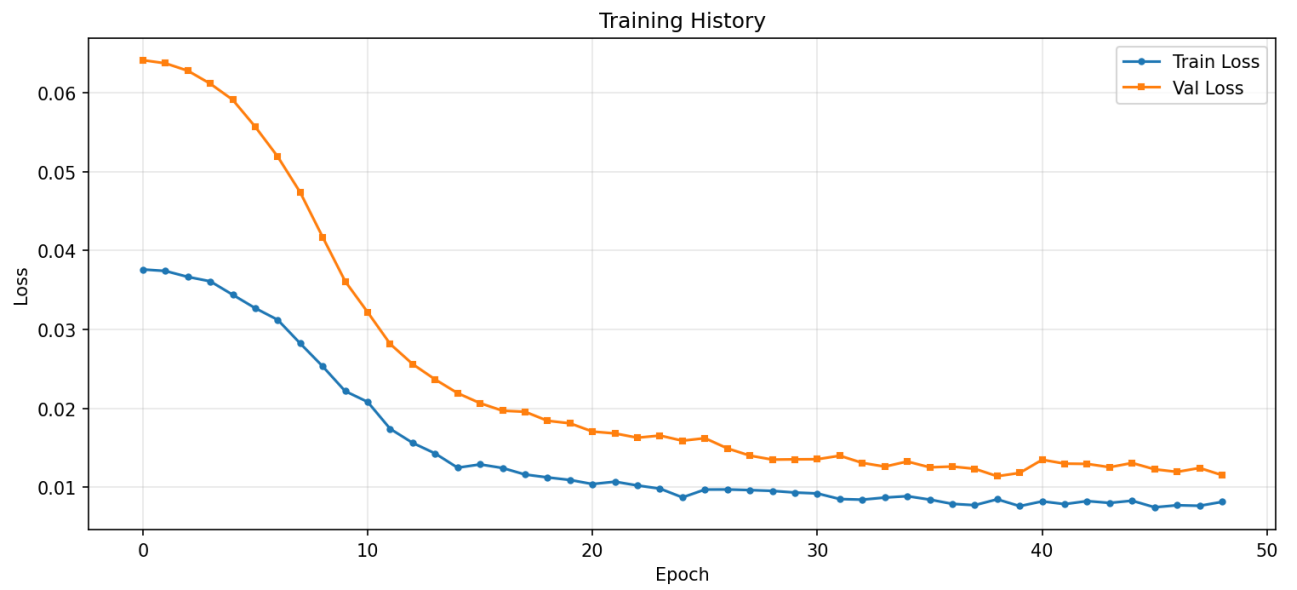
4.4 评估指标•丁乙

1. 均方根误差 (RMSE)
2. 平均绝对误差 (MAE)
3. 加权 R^2 分数: 考虑不同目标重要性的决定系数
4. 验证损失: 加权MSE损失, 监控训练过程
5. 公榜PB

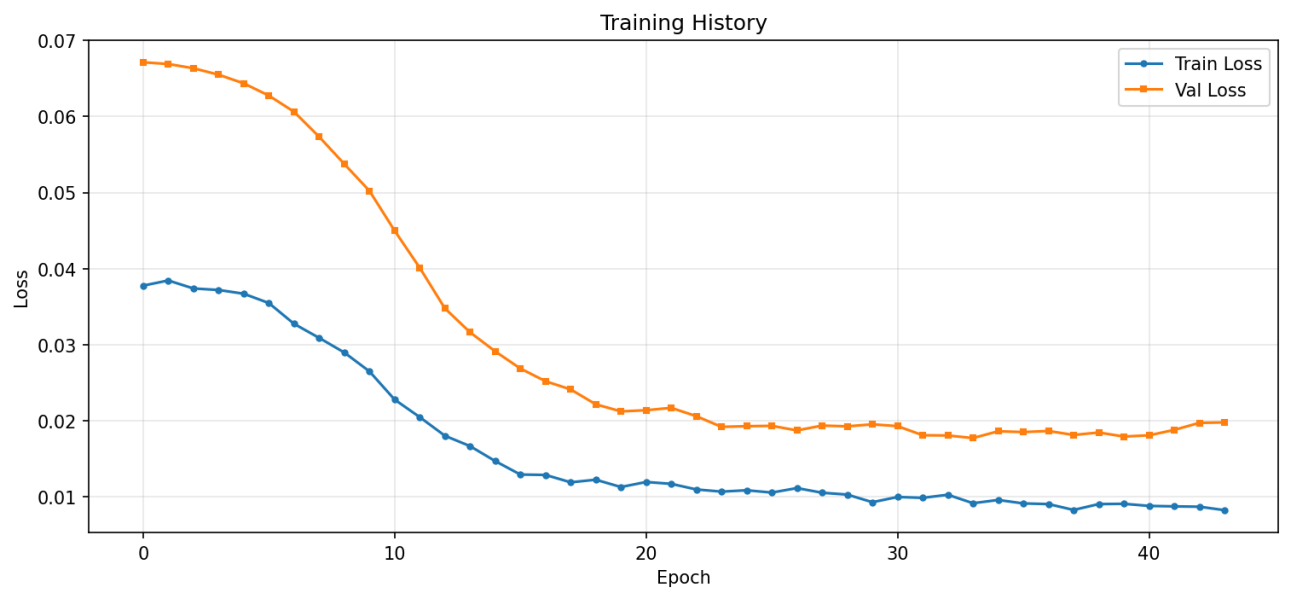
五、实验记录和结果分析 (此部分内容需要标识出来具体是小组哪位成员完成的)

运行train.py输出5折训练日志和每一折的Loss折线图。

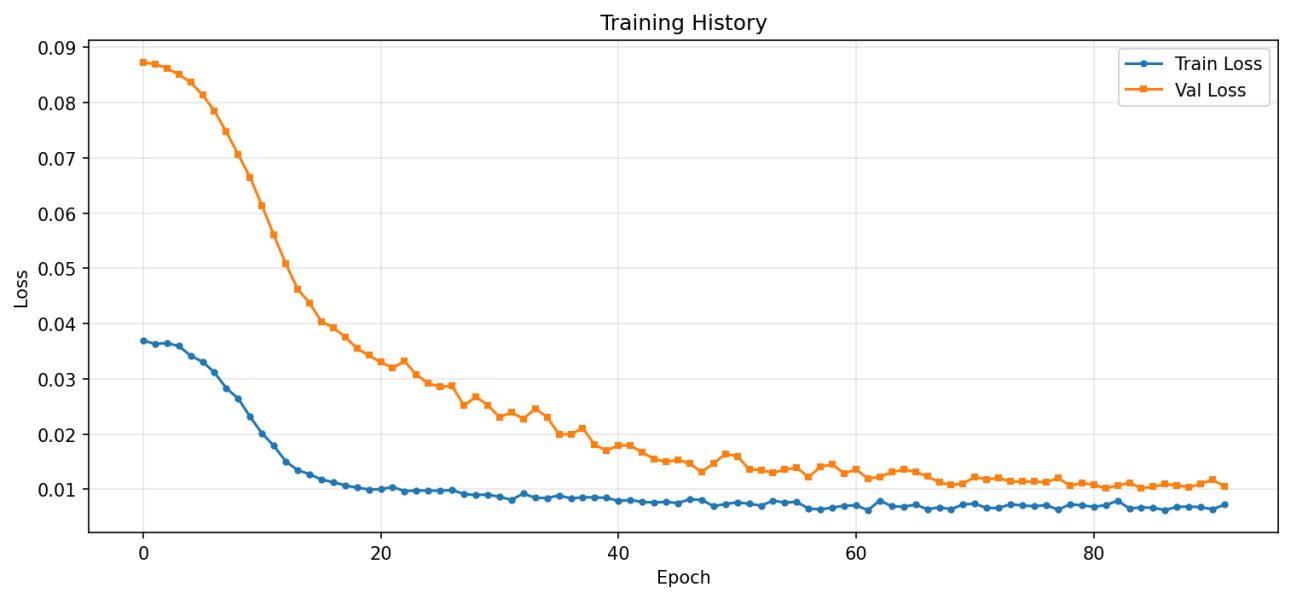
第0折:



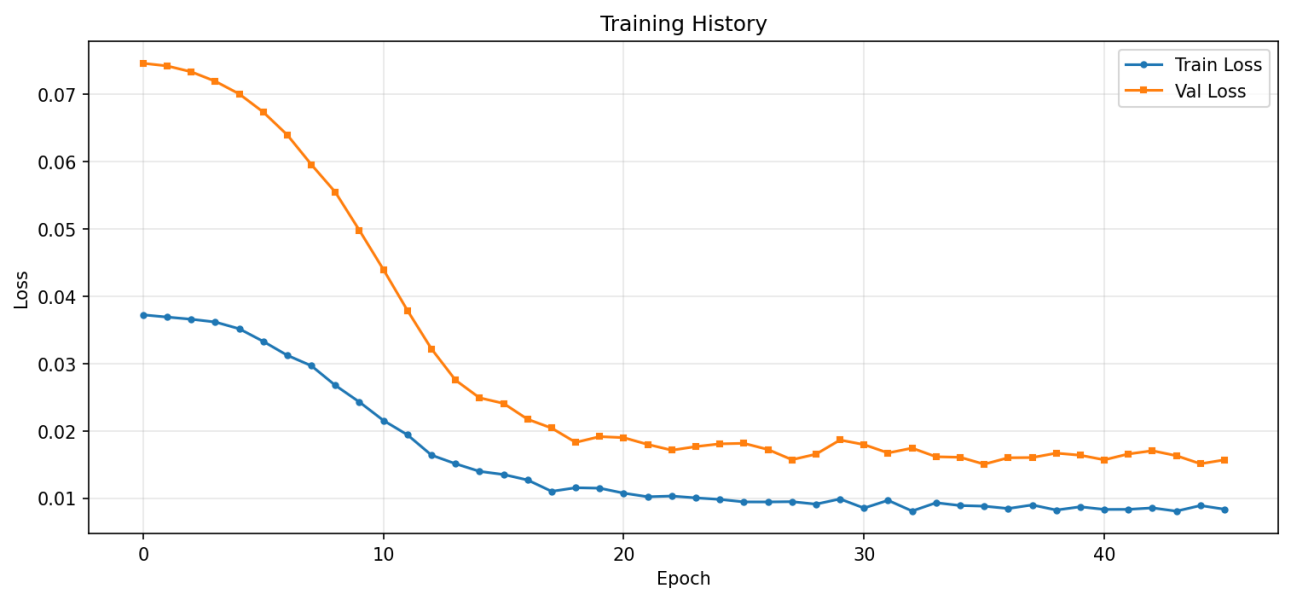
第1折:



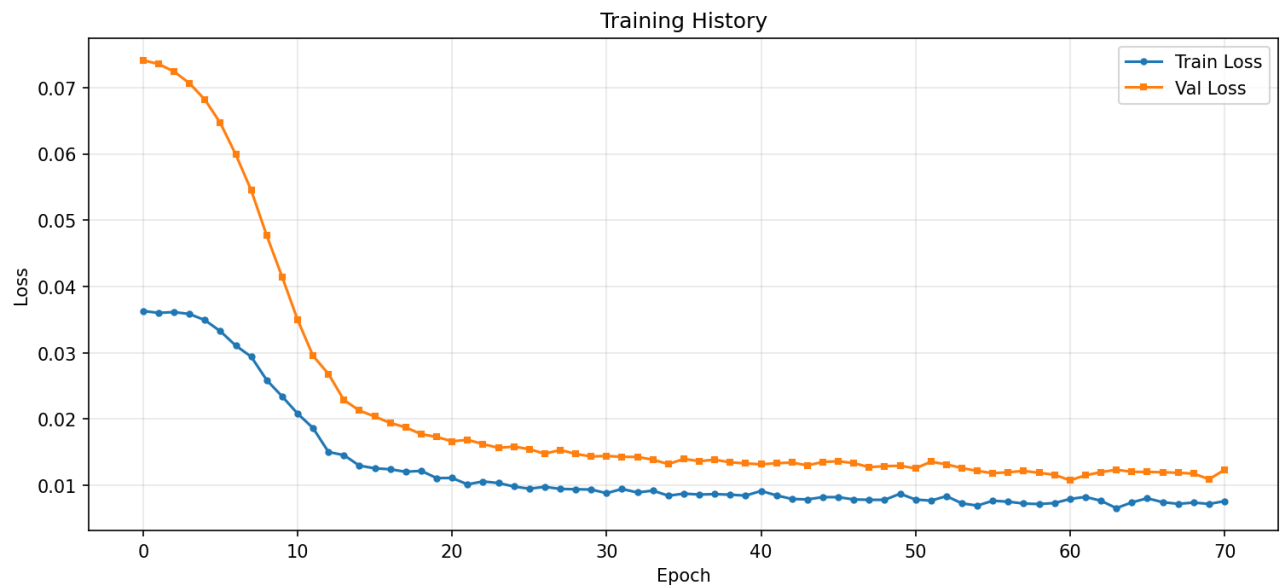
第2折:



第3折:



第4折:



5.1 收敛趋势分析•俞骞

根据对5折交叉验证的训练输出日志和上面的Loss折线图分析，所有折的训练过程都呈现出下面的相似模式：

1. **初始阶段（Epoch 1-10）**：训练损失从约0.08迅速下降至0.01-0.02区间，验证损失同步下降， R^2 指标从负值（-1.2~-0.7）改善至负值区间高位，说明模型快速学习数据基本模式
2. **中期阶段（Epoch 11-30）**：训练损失稳步下降0.01以下，验证损失出现波动但总体呈下降趋势， R^2 指标转正并在0.6-0.75区间波动，表明模型开始产生有效预测
3. **后期阶段（Epoch 31-结束）**：训练损失进一步下降至60-80区间，验证损失趋于稳定，最佳验证损失出现在各折的35-42轮，早停机制在46-53轮触发，防止过拟合

5.2 性能指标分析•俞骞

1. **R^2** ：训练集上 R^2 随训练轮次逐步提升，而验证集上 R^2 从负值稳步提升至正值，最终稳定在0.70-0.75，这说明模型较为有效的解释了目标变量
2. 公榜分数为0.65，排名约900+/3158

Output Data

submission.csv (236 B)

sample_id	target
ID1001187975__Dry_Cl over_g	0.020954562351107597
ID1001187975__Dry_De ad_g	20.149606704711914
ID1001187975__Dry_Gr een_g	28.419219970703125
ID1001187975__Dry_To tal_g	49.001766204833984
ID1001187975__GDM_g	28.6668643951416

Notebook Input Output Logs Comments (0)



Competition Notebook

[CSIRO - Image2Biomass Prediction](#)

Public Score

0.65

Best Score

0.65 V41

```
In [1]: import os
```

5.3 消融实验•丁乙

通过对数据核心机制进行消融实验得到结果，可以得到以下结论：

1. 多任务学习：对比多任务学习（除了生物量还包含所有辅助任务（NDVI回归、草高回归、地区分类、草种分类））和单任务学习（仅训练生物量预测）的结果表明多任务框架下训练更稳定，验证损失波动减小，这可能是由于辅助任务提供额外梯度信号，加速早期收敛
2. Mixup数据增强：对比仅使用原始数据进行训练，Mixup数据增强明显降低了训练集与验证集之间的性能差距

六、结论

俞骞：

在本次项目中，我主要负责模型结构设计和实验验证工作，这段经历让我深刻体会到深度学习模型从理论设计到实际落地之间的复杂过程。最初我选择了 EfficientNet 作为主干网络，然而在实际训练中，我很快就遇到了训练时间过长的的问题，训练 1 折就需要 1 小时左右。这使我考虑模型的效率问题：在参数量、计算复杂度和预测精度之间进行取舍。我尝试了不同的架构模型，最终决定选择 Convnext_Base 模型。

实验过程中最让我印象深刻的是那些微小的选择会产生相当巨大的影响。比如 Mixup 数据增强的 α 参数，从 0.2 调整到 0.4，就让验证损失降低了近 8 个百分点；学习率调度策略从 CosineAnnealing 切换到 ReduceLROnPlateau 后，模型在后期训练中表现得更加稳定。

丁乙：

在本次项目的算法实现过程中，我主要负责数据划分策略的验证与优化。这段经历让我深刻认识到，在深度学习项目中，数据是模型的基石，而数据划分策略则是影响模型泛化能力的关键“阀门”。在阅读了比赛讨论区关于防止数据泄露的讨论后，我决定摒弃简单的随机划分，转而采用“分层分组交叉验证”策略，即以 State 为分层目标，以 Sampling_Date 为分组依据，确保同一采样日期的样本不会同时出现在训练集和验证集中。

然而，这个看似完美的方案在实际执行中遇到了挑战。与讨论区中很多参赛者反馈的情况类似，我最初实现的划分导致了严重的过拟合，交叉验证（CV）分数与排行榜（LB）分数之间出现了巨大鸿沟（例如 CV 可达 0.70 而 LB 仅为 0.54）。我仔细排查了原因，发现问题出在数据分布的潜在偏差上：不同州（State）的样本分布差异巨大，而测试集可能在某些州的代表性上存在偏差。仅仅按 Sampling_Date 分组，可能无意中某些州的特有模式泄露到了验证集，导致 CV 分数虚高，但模型并未学到真正通用的特征。

为解决这个问题，我进行了更精细的探索。我尝试将分层目标 State 与更精细的分组目标（如 Sampling_Date 的月份）结合，并使用不同的随机种子进行多次划分，以寻找更稳定、更具代表性的数据子集。最终，我采用了一种兼顾分层与分组的“种子搜索”策略，并辅以更强的数据增强作为正则化手段。这个优化过程使我的 CV 分数从“虚高”的 0.75 以上，稳定到了 0.67 左右，并且与 LB 分数的相关性显著增强（CV 约

0.67, LB 约 0.65)。

此次探索让我深刻体会到，算法落地不仅关乎模型本身，更关乎对数据的深刻理解。数据划分策略并非一成不变的预设，而是需要根据数据内在结构（如时空相关性、类别分布）进行动态调整的实验过程。从简单分组到分层分组，从单一种子到多种子搜索，每一次调整都像在调整一个精密的校准仪，目标是在模型复杂度和数据多样性之间找到一个平衡点，从而让验证分数真正成为模型泛化能力的“可靠标尺”。最终，这种对数据划分的精细化把控，与模型架构、损失函数、训练策略一同，构成了我们算法稳健性的重要支柱。

张建夫

此次项目让我深刻认识到，深度学习不仅仅是理论架构的堆砌，更是对实际问题需求的精准把握与耐心调优。每一次看似微小的参数调整背后，都蕴含着对模型行为本质的洞察——从 EfficientNet 转向 ConvNeXt_Base，表面是模型选择的权衡，实则是计算效率与预测精度的平衡；Mixup 的 α 参数和调度策略的优化，则体现了在模型训练的“拐点”上。最终，这种从试错到认知、从架构到参数的完整探索过程，实现了更加高效的模型。

七、代码附录

多任务头初始化代码：

```
# Feature extractor - shared across all tasks
self.feature_extractor = nn.Sequential(
    nn.Linear(feat_dim, 1024),
    nn.BatchNorm1d(1024),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(1024, 512),
    nn.BatchNorm1d(512),
    nn.ReLU(),
    nn.Dropout(0.3)
)

# Main task. Biomass regression head
self.biomass_head = nn.Sequential(
    nn.Linear(512, 256),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, num_biomass_targets)
)
```

```
if use_multitask:
    # NDVI regression head
    self.ndvi_head = nn.Sequential(
        nn.Linear(512, 128),
        nn.ReLU(),
        nn.Linear(128, 1)
    )

    # Height regression head
    self.height_head = nn.Sequential(
        nn.Linear(512, 128),
        nn.ReLU(),
        nn.Linear(128, 1)
    )

    # State classification head
    self.state_head = nn.Sequential(
        nn.Linear(512, 128),
        nn.ReLU(),
        nn.Linear(128, num_states)
    )

    # Species classification head
    self.species_head = nn.Sequential(
        nn.Linear(512, 128),
        nn.ReLU(),
        nn.Linear(128, num_species)
    )

self.freeze_backbone()
```

多任务学习代码:


```

# 多任务学习
if Config.USE_MULTITASK:
    outputs = model(images, return_auxiliary=True)

    # 计算生物量损失
    if targets_b is not None:
        # mixup损失函数
        biomass_loss = mixup_criterion(criterion_biomass, outputs['biomass'], targets_a, targets_b, lam)
    else:
        biomass_loss = criterion_biomass(outputs['biomass'], targets_a)

    # 计算辅助任务损失（不对辅助任务使用mixup）
    aux_loss = 0
    # NDVI回归损失
    aux_loss += criterion_aux_reg(outputs['ndvi'], auxiliary_targets['ndvi'].to(Config.DEVICE))
    # 草高回归损失
    aux_loss += criterion_aux_reg(outputs['height'], auxiliary_targets['height'].to(Config.DEVICE))
    # 地区分类损失
    aux_loss += criterion_aux_cls(outputs['state'], auxiliary_targets['state'].to(Config.DEVICE).squeeze())
    # 草种分类损失
    aux_loss += criterion_aux_cls(outputs['species'], auxiliary_targets['species'].to(Config.DEVICE).squeeze())
    # 平均辅助损失
    aux_loss = aux_loss / 4

    # 总损失 = 生物量损失 + 权重系数 × 辅助损失
    loss = biomass_loss + Config.AUXILIARY_WEIGHT * aux_loss
    biomass_loss_total += biomass_loss.item()
    aux_loss_total += aux_loss.item()

```

单任务学习代码:

```

# 单任务学习
else:
    outputs = model(images, return_auxiliary=False)

    if targets_b is not None:
        loss = mixup_criterion(criterion_biomass, outputs, targets_a, targets_b, lam)
    else:
        loss = criterion_biomass(outputs, targets_a)
    biomass_loss_total += loss.item()

```

模型验证代码:

```

model.eval()
total_loss = 0
all_preds = []
all_targets = []

with torch.no_grad():
    for images, biomass_targets, _ in loader:
        images = images.to(Config.DEVICE)
        biomass_targets = biomass_targets.to(Config.DEVICE)

        outputs = model(images, return_auxiliary=False) # 只预测生物量
        loss = criterion(outputs, biomass_targets)

        total_loss += loss.item()
        all_preds.append(outputs.cpu().numpy())
        all_targets.append(biomass_targets.cpu().numpy())

all_preds = np.vstack(all_preds)
all_targets = np.vstack(all_targets)

# 均方根误差
rmse = np.sqrt(mean_squared_error(all_targets, all_preds))
# 平均绝对误差
mae = mean_absolute_error(all_targets, all_preds)

# 计算加权R²
r2 = ut.weighted_r2_score(all_targets, all_preds, Config.TARGET_WEIGHTS)

return total_loss / len(loader), rmse, mae, r2

```

训练函数:

```

for fold_idx, (train_idx, val_idx) in enumerate(gkf.split(train_wide, groups=groups)):
    if fold_idx not in folds_to_train:
        continue

    print("\n" + "="*60)
    print(f"Training Fold {fold_idx + 1}/{Config.NUM_FOLDS}")
    print("="*60)

    train_data_raw = train_wide.iloc[train_idx].reset_index(drop=True)
    val_data_raw = train_wide.iloc[val_idx].reset_index(drop=True)

    # 对数据进行拟合和转换（包括标准化、编码等）
    train_data, val_data, scalers, label_encoders = dl.fit_and_transform_fold_data(
        train_data_raw, val_data_raw
    )

    # 创建预处理信息字典
    preprocessing_info = {
        'scalers': scalers, # 标准化器（用于逆变换）
        'label_encoders': label_encoders, # 标签编码器
        'num_states': len(label_encoders['State'].classes_) + 1, # 地区类别数
        'num_species': len(label_encoders['Species'].classes_) + 1, # 草种类别数
        'target_names': Config.TARGET_NAMES # 目标名称
    }
    print(f"Train size: {len(train_data)}, Val size: {len(val_data)}")
    # 保存预处理信息到文件
    with open(os.path.join(Config.OUTPUT_DIR, f'preprocessing_info_fold{fold_idx}.pkl'), 'wb') as f:
        pickle.dump(preprocessing_info, f)

```

加权mse计算代码:

```
def forward(self, inputs, targets):  
    # Calculate the squared error  
    squared_error = (inputs - targets) ** 2  
  
    # Apply the element-wise weights  
    weighted_squared_error = squared_error * self.weights  
  
    # Calculate the mean across the batch and targets  
    loss = torch.mean(weighted_squared_error)  
  
    return loss
```

最佳模型比较与保存:

```
if val_loss < best_val_loss:  
    best_val_loss = val_loss  
  
    # 保存模型检查点  
    checkpoint = {  
        'model_state_dict': model.state_dict(),  
        'config': {  
            'model_name': Config.BACKBONE_MODEL,  
            'num_biomass_targets': len(Config.TARGET_NAMES),  
            'num_states': preprocessing_info['num_states'],  
            'num_species': preprocessing_info['num_species'],  
            'use_multitask': Config.USE_MULTITASK  
        },  
        'epoch': epoch,  
        'val_loss': val_loss,  
        'val_rmse': val_rmse,  
        'val_mae': val_mae,  
        'val_r2': val_r2  
    }  
    # 保存最佳模型  
    torch.save(checkpoint, os.path.join(Config.OUTPUT_DIR, f'best_model_fold{fold_idx}.pth'))  
    print("Best model saved!")  
    patience_counter = 0
```